Instructor Handout 6: Process State Diagram, Schedulers, and Context Switch

Lecture 6 — How the OS Tracks and Switches Processes

Auqib Hamid Lone
Department of Computer Science & Engineering, GCET Ganderbal

October 29, 2026

Instructor Copy — Not for Student Distribution

Concepts That Click

6.1 Before You Teach This Lecture

- **Silberschatz, Galvin & Gagne, *Operating System Concepts*, 8e** — Ch. 3 §3.1 (process states, p. 103) and §3.2 (process scheduling: queues, schedulers, context switch, pp. 105–110). The primary anchor for the five-state diagram, the queueing diagram, the three schedulers, the I/O-bound/CPU-bound mix, and the context switch. *Page-verified* via the course index.
- **Stallings, *Operating Systems: Internals and Design Principles*, 6e** — Ch. 3 §3.2 (two-, five-, and seven-state models, p. 111) for suspended processes, and §3.4 (process control, p. 135) for the step list of a process switch and the mode-switch/process-switch distinction. *Page-verified.*
- **Bovet & Cesati, *Understanding the Linux Kernel*, 3e** — Ch. 3 (processes, the process switch, pp. 79–130). Optional depth on how Linux actually saves and restores context. *Page-verified at chapter level.*
- **Silberschatz et al., Ch. 5 §5.1 (from p. 183)** — skim the dispatcher paragraph; the deck previews it and Lecture 8 develops it.
- **Draw Figures 6.1 and 6.4 from memory** before class. If you hesitate on the missing arrows (`waiting` → `running`, `ready` → `waiting`), re-read p. 103.
- **Continuity.** Lecture 5 introduced the process and the PCB; Lecture 3 introduced the trap and the mode switch. This lecture uses both. Lecture 7 (process creation) and Lectures 8–9 (scheduling algorithms) build on today's queues and short-term scheduler — do not teach FCFS/RR today.

Teaching note: The running example is one single-core lab machine: `P1` = `gcc` (CPU-bound), `P2` = an editor (I/O-bound), `P3` = a backup job in the batch queue. Write these three lines in a corner of the board at the start and leave them up all hour; every example refers back to them.

6.2 Opening: One CPU, Three Programs

Ask: the machine has one core, and `gcc`, the editor, and a queued backup all "feel" running — how many are actually running at this instant? Exactly one. The rest of the hour explains how the kernel tracks the others: the states are the vocabulary, the queues are the data structures, the schedulers are the policies, and the context switch is the mechanism. Recall from Lecture 5 that the PCB holds the state, `PC`, registers, scheduling and memory-management information, accounting, and open files, and that it is what lets a process stop and later resume. Today's question is about one PCB field: which values can *state* hold, which events change it, and who changes it.

> **Teaching note:** Students tend to think "running" means "open on my screen." The one-core framing breaks that: an open editor is almost never running. Get them to say it aloud before you draw anything.

6.3 The Process State Diagram

Definition (Process state). *The state of a process is the PCB field that records its current activity. A process is in exactly one of five states at any instant:* ***`new`*** *(being created),* ***`ready`*** *(in memory, waiting only for a CPU),* ***`running`*** *(executing on a CPU),* ***`waiting`*** *(blocked until an event such as I/O completion or a signal), and* ***`terminated`*** *(finished; resources being reclaimed).*

Stallings calls `waiting` "blocked" and `terminated` "exit" (Ch. 3 §3.2, p. 111); the states are the same.

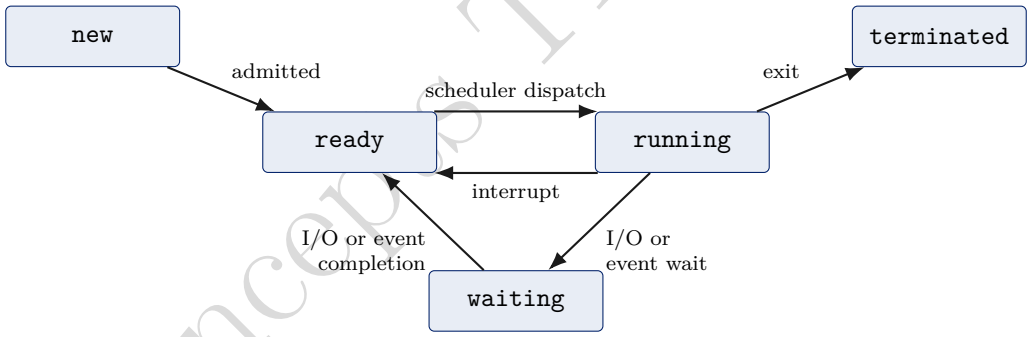


Figure 6.1: The five-state diagram — after Silberschatz et al., Ch. 3 §3.1, p. 103.

How to present it. Follow the coordinate map in the deck. Draw `ready` and `running` first, side by side in the middle of the board, with the dispatch arrow above and the interrupt arrow below — this pair is the heart of the lecture. Add `waiting` underneath, with the two diagonal arrows (I/O or event wait going down from `running`; completion going up into `ready`). Add `new` at upper left and `terminated` at upper right last. As you draw each arrow, say its triggering event and its agent: admitted (long-term scheduler), dispatch (short-term scheduler plus dispatcher), interrupt (kernel handler), I/O or event wait (the process itself, by a system call), completion (kernel interrupt handler), exit (the process or the kernel).

Then point at the two places where arrows are *missing*. There is no `waiting` → `running`: a woken process must queue for the CPU. There is no `ready` → `waiting`: only a running process can issue the request it would wait on. Keeping "wake up" and "get the CPU" separate is what lets the scheduler enforce a policy.

Concepts That Click

Teaching note: The most common error is drawing a direct arrow from `waiting` to `running`, especially when "the CPU is idle anyway." Ask the deck's Socratic question before you draw the completion arrow: *`P2` is waiting for the disk, the interrupt arrives, and the CPU is idle — does `P2` go straight to running?* Let them vote, then answer: no, `waiting` → `ready`, and the scheduler then picks among all ready processes, perhaps `P2` almost at once.

Board: the editor for one keystroke (deck example). *On the board, in order:* (i) put a token labelled `P2` on `running`; (ii) `P2` calls `read` on the keyboard, no key pending — move the token to `waiting` and write "keyboard queue"; (iii) a key is pressed — move it to `ready` and write "does NOT run yet"; (iv) the scheduler picks it — move it to `running`; (v) it echoes and calls `read` again — back to `waiting`. *Say:* "Three transitions per keystroke, and the editor spends almost all its life in waiting."

Board: counting processes per state (deck example plus a second instance). *On the board, in order:* (i) write "4 cores, 40 processes, 9 ready"; (ii) write "running $\leq$ 4"; (iii) ask the class for the number waiting; (iv) write $40 - 4 - 9 = 27$ (waiting, or new/terminated). Second instance, for practice: "8 cores, 120 processes, all cores busy, 17 ready, 2 new" gives $120 - 8 - 17 - 2 = 93$ waiting. *Say:* "With N cores at most N are running; everything else is queued somewhere."

Suspended processes: the seven-state model

Memory is finite. If every in-memory process is waiting, the CPU idles even though more work is queued on disk. The OS can **swap out** a process — copy its image to disk and free its memory — leaving it **suspended**. Stallings splits `ready` and `waiting` each into an in-memory and a suspended version: `ready/suspend` (on disk, runnable once brought back) and `blocked/suspend` (on disk and still waiting).

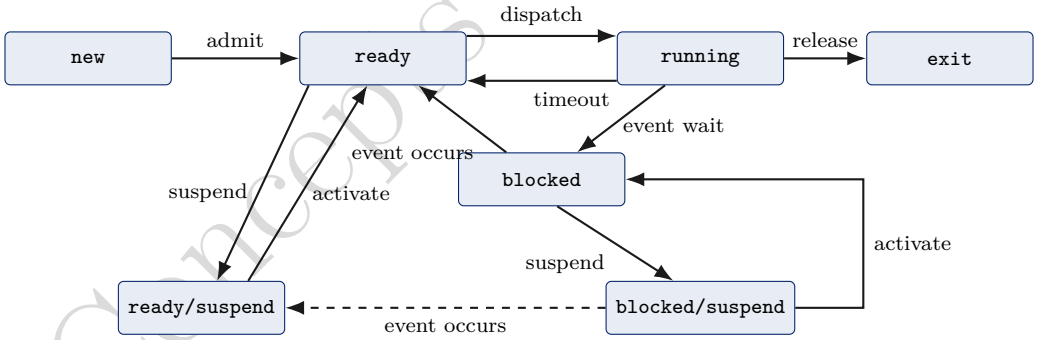


Figure 6.2: The seven-state model — after Stallings, Ch. 3 §3.2, p. 111. Top row in memory; bottom row swapped out.

How to present it. Redraw the five-state row along the top (`new`, `ready`, `running`, `exit`) with `blocked` beneath it; then draw a horizontal line labelled "disk" and add the two suspended states below it. Draw the suspend/activate pairs last, and finish with the dashed "event occurs" arrow from `blocked/suspend` to `ready/suspend`.

Board: trace a suspended process. *On the board, in order:* (i) a process is `blocked` on the disk; (ii) memory is short — suspend: `blocked/suspend`; (iii) the disk completes while it is on disk — `ready/suspend`, *not* `ready`; (iv) activate — `ready`. Ask before step (iii): "its event happened, so is it ready?" *Say:* "Ready means in memory and runnable. On disk it is still suspended."

Concepts That Click

Board: choosing whom to suspend. Memory holds three images: `P1` just blocked on the network, `P2` blocked on the disk (done in a few ms), `P3` blocked on the keyboard, and `P4` is `ready/suspend`. Ask which to swap out. The sensible answer is `P3`, waiting on a human: `P3` → `blocked/suspend`, `P4` → `ready`. This is an application of the deck's model, not a textbook policy; present it as reasoning, not as a rule.

6.4 Scheduling Queues

Definition (Scheduling queues). *The job queue holds all processes in the system (in Silberschatz's batch-oriented description, jobs awaiting admission); the ready queue holds the in-memory* ***`ready`*** *processes, usually as a linked list of PCBs; each device has a device queue of processes* ***`waiting`*** *for it.*

A queue header holds pointers to the first and last PCB, and each PCB has a next-pointer. Moving a process between queues moves a pointer, not the process, and a PCB sits on exactly one queue at a time (Silberschatz et al., Ch. 3 §3.2, pp. 105–110).

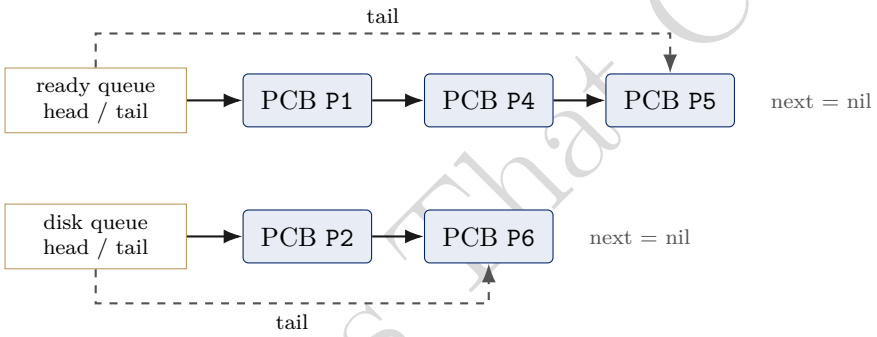


Figure 6.3: Ready queue and disk queue as linked lists of PCBs.

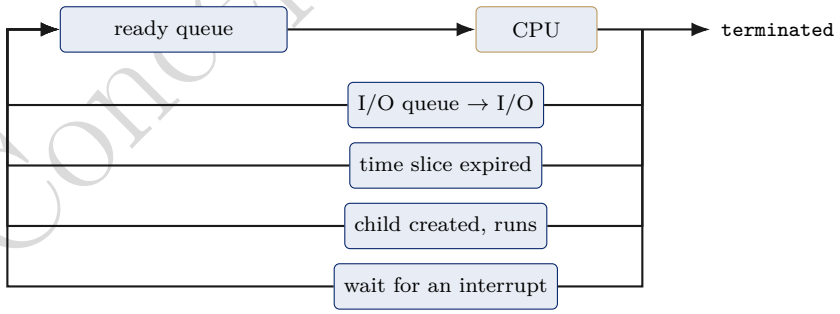


Figure 6.4: The queueing diagram — after Silberschatz et al., Ch. 3 §3.2, pp. 105–110.

How to present it. For Figure 6.4, draw the ready queue and the CPU on the top line first, then the exit arrow. Then add the four return rows one by one, each time naming the reason a running process leaves the CPU: an I/O request (device queue), time slice expired, a child created, waiting for an interrupt or event. Close the loop back into the tail of the ready queue last. *Say:* "Every path out of the CPU except termination leads back here." Tie each route to a process: `P2`

Concepts That Click

leaves by the I/O route almost every burst; `P1` by the time-slice route; a shell running `gcc` by the child route.

> **Teaching note:** Students put waiting processes "at the back of the ready queue." Correct it with Figure 6.3: the waiting `P2` is on the disk queue, and its single next-pointer is in use there. It cannot also be in the ready list.

Board: queue membership over time (deck example). Draw four columns — Running, Ready queue, Keyboard queue — and fill one row per event: t_0 start (`P1`; `P2`; —); t_1 `P1`'s slice expires (`P2`; `P1`; —); t_2 `P2` calls `read` (`P1`; —; `P2`); t_3 key pressed (`P1`; `P2`; —); t_4 slice expires (`P2`; `P1`; —); t_5 `P3` admitted (`P2`; `P1`, `P3`; —). Pause at t_3 and ask who runs now: `P1` keeps the CPU until t_4. *Check aloud:* one running process per row; each PCB in exactly one place.

Board: a second trace with a disk queue (the assignment's Q4 is a longer version; do not solve that one in class). Slice 5 ms; at $t = 0$ `P1` running, ready `P2`, `P3`. $t = 5$: slice expires — running `P2`, ready `P3`, `P1`. $t = 6$: `P2` reads the disk — running `P3`, ready `P1`, disk `P2`. $t = 8$: disk completes — running `P3`, ready `P1`, `P2`. Ask the class to predict the row for $t = 8$ before you write it; the trap is putting `P2` in front of `P1`.

6.5 Schedulers

Three schedulers, three timescales (Silberschatz et al., Ch. 3 §3.2, pp. 105–110):

- **Long-term (job) scheduler**: job pool → ready queue (`new` → `ready`); runs every few seconds to minutes; controls the **degree of multiprogramming** n. In a stable system the admission rate equals the departure rate. Batch systems have one; time-sharing systems such as UNIX and Windows often have none.
- **Short-term (CPU) scheduler**: ready queue → CPU; runs every few milliseconds (at least once per time slice and after every block); must be fast because its running time is pure overhead.
- **Medium-term scheduler**: swaps processes out to reduce n and back in to continue where they left off; used to improve the mix or free memory.

The long-term scheduler's key decision is the **process mix**. An I/O-bound process (`P2`) has many short CPU bursts; a CPU-bound one (`P1`) has few long ones. All I/O-bound: the ready queue is nearly always empty and the CPU idles. All CPU-bound: the device queues are empty and the devices idle.

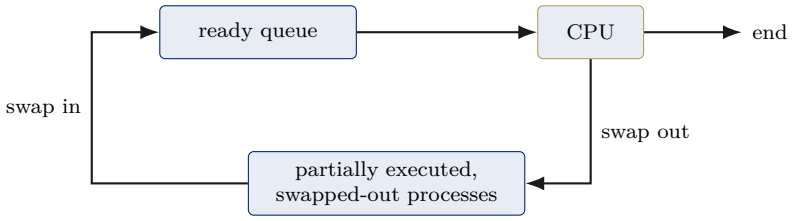


Figure 6.5: The medium-term scheduler's swap loop — after Silberschatz et al., Ch. 3 §3.2, pp. 105–110.

Concepts That Click

Schedulers on the state diagram. Go back to Figure 6.1 and colour the arrows: long-term drives `new` → `ready`; short-term drives `ready` → `running` and, with preemption, `running` → `ready`; medium-term drives the suspend/activate arrows of Figure 6.2. Leave `running` → `waiting` (the process's own request) and `waiting` → `ready` (the completion interrupt) uncoloured: no scheduler chooses them.

Board: scheduler overhead (deck example). *On the board, in order:* (i) draw a bar of 1 ms labelled "decide" followed by 10 ms labelled "run"; (ii) write $1/(10+1)$; (iii) write $\approx 9\%$. *Say:* "That is why the short-term scheduler must be fast."

Board: the same formula backwards. Ask: what slice keeps scheduling overhead at most 4% with a 1 ms decision? $1/(q+1) \le 0.04$ gives $q \ge 24$ ms. (This is GATE practice Q8 — use it in class only if you are not setting that sheet.)

> **Teaching note:** Live question (deck Socratic Check 2): students open dozens of browser tabs until a Linux laptop crawls — which scheduler should have stopped it, and why did none? Answer: admission control is the long-term scheduler's job, but a time-sharing system typically has none; memory pressure is handled by swapping/paging instead, so the machine slows rather than refuses.

6.6 The Context Switch

Definition (Context switch). *The context of a process is the CPU and kernel state needed to resume it — registers, PC, process state, memory-management information — all represented in its PCB. A context switch saves the context of the current process into its PCB and restores the saved context of the process scheduled to run next (Silberschatz et al., Ch. 3 §3.2, p. 110).*

The switch has two halves, a state save and a state restore, and is triggered by an interrupt, a trap, or a system call that blocks.

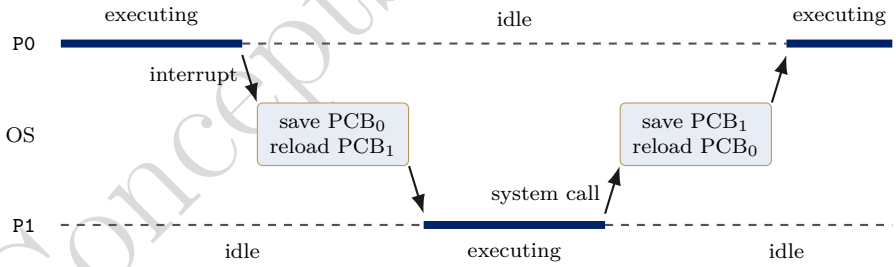


Figure 6.6: A context switch on a timeline — after Silberschatz et al., Ch. 3, pp. 103–110.

How to present it. Draw three horizontal lanes (`P0`, OS, `P1`). Thicken `P0`'s lane for its executing segment, draw the interrupt arrow down to the first OS box ("save PCB_0, reload PCB_1"), then the arrow down to `P1`'s executing segment. Repeat in reverse for the system call. Dash the idle segments last. *Say:* "In the gold boxes, nobody's program is running. That is overhead."

The steps (Stallings, Ch. 3 §3.4, p. 135; Bovet & Cesati, Ch. 3): (1) an interrupt or trap enters the kernel and the hardware saves the user `PC` and flags; (2) the kernel saves `P0`'s remaining registers into PCB_0; (3) it updates PCB_0's state (`ready` or `waiting`) and links it onto the right queue; (4) the short-term scheduler selects `P1`; (5) the kernel sets PCB_1 to `running` and switches memory-management state (page-table base, TLB as needed); (6) it restores `P1`'s registers and returns to user mode at `P1`'s saved `PC`.

Concepts That Click

The **dispatcher** is the module that does this: switch context, switch to user mode, jump to the right location. Scheduler = policy, dispatcher = mechanism; the time to stop one process and start another is the **dispatch latency** (Silberschatz et al., Ch. 5 §5.1, from p. 183; developed in Lecture 8).

A **mode switch** (user → kernel → user for the same process, e.g. `getpid`) is not a context switch. Every context switch includes mode switches; most mode switches cause no context switch. The test: *does the caller keep the CPU when the kernel returns?*

Cost: switching is pure overhead; it depends on memory speed, the number of registers, and special instructions; it is typically a few microseconds plus cold-cache and TLB effects; some CPUs keep multiple register sets so a switch just changes a pointer. The more complex the OS and its memory management, the more each switch costs. With switch cost s and slice q, the overhead fraction is $s/(q+s)$.

> **Teaching note:** Students equate "entered the kernel" with "switched process." Ask the deck's Socratic Check 3 before defining the mode switch: a process calls `getpid`, and later `read` on an empty pipe — which causes a context switch? Only the second, because the caller blocks.

Board: the overhead table (deck example). Write $s = 5$ μs and four rows: $q = 10$ ms gives $5/10005 \approx 0.05\%$; $q = 1$ ms gives $5/1005 \approx 0.50\%$; $q = 100$ μs gives $5/105 \approx 4.76\%$; $q = 10$ μs gives $5/15 \approx 33.3\%$. Before the last row, ask the class to guess. *Say:* "Shrink the slice and the CPU spends more of its life switching than working."

Board: `P1` and `P2` on one core (deck example). $t = 0$: `P1` running, `P2` waiting. $t = 3$ ms: key pressed — `P2` → ready, mode switch only, `P1` resumes. $t = 10$ ms: slice expires — switch #1, `P1` → ready, `P2` → running. $t = 10.2$ ms: `P2` echoes and reads again — switch #2, `P2` → waiting, `P1` → running. Count: two context switches, three kernel entries; $2 \times 5 = 10$ μs, about 0.1% of 10.2 ms.

Board: a third overhead instance for practice. $s = 2$ μs: at what q is overhead exactly 10%? $s/(q+s) = 0.1 \Rightarrow q = 9s = 18$ μs.

6.7 Putting It Together

Walk `P3` through its whole life, pointing at both diagrams as you go: submitted (`new`, job queue); admitted by the long-term scheduler (`ready`); dispatched (`running`); writes to the backup disk (`waiting`, disk queue, context switch away); swapped out by the medium-term scheduler (`blocked/suspend`); write completes (`ready/suspend`); swapped in (`ready`); dispatched, finishes, `exit` (`terminated`). All three schedulers appear in one life.

Close on the four misconceptions from the deck: a woken process does not run immediately; most system calls are mode switches, not context switches; time-sharing systems often have no long-term scheduler; waiting processes are on device queues, not the ready queue.

6.8 Exit Ticket

1. Name the two transitions missing from the five-state diagram and why each is missing.
2. Which scheduler controls the degree of multiprogramming, and does Linux have one?
3. $s = 5$ μs, $q = 45$ μs: overhead? *Answer:* 10%.
4. `getpid` vs. `read` on an empty pipe: which causes a context switch? *Answer:* the `read`.

Concepts That Click

6.9 Summary: One-Page Refresh

- Five states; six legal transitions; no `waiting` → `running`, no `ready` → `waiting`. Suspension adds `ready/suspend` and `blocked/suspend` (Stallings, p. 111).
- Job queue, ready queue, device queues; PCBs linked through a next-pointer; one queue per PCB at a time.
- Long-term: admission, n, mix. Short-term: next process, every few ms, must be fast. Medium-term: swapping.
- Context switch = save PCB of current + restore PCB of next; pure overhead $s/(q+s)$; dispatcher is the mechanism.
- Mode switch $\neq$ context switch.

6.10 Readings to Prepare Next Time

- **Silberschatz et al., Ch. 3 §3.3** — operations on processes: creation, termination, `fork`/`exec` (from p. 110). *(Page-verified.)*
- **Stallings, Ch. 3 §3.4** — process control and creation (p. 135). *(Page-verified.)*
- **Bovet & Cesati, Ch. 3** — creating and destroying processes (pp. 79–130). *(Page-verified at chapter level.)*

Concepts That Click